

Documentación del sistema de detección con YOLO para señales de tránsito y obstáculos

Objetivos

Los objetivos principales del proyecto fueron:

- Detectar señales de tránsito personalizadas.
- Detectar semáforos.
- Utilizar un modelo preentrenado para detectar obstáculos.
- Combinar ambos modelos en un solo pipeline.
- Implementar un sistema de prioridades.
- Reducir parpadeos y falsas detecciones.
- Preparar el sistema para integración futura con ROS2 y control del robot.
-

Tecnologías utilizadas

Ubuntu

Se utilizó Ubuntu como sistema operativo principal debido a que es ampliamente usado en robótica y visión computacional.

Python

Python fue utilizado como lenguaje principal porque tiene integración directa con OpenCV y Ultralytics YOLO.

OpenCV

OpenCV para:

- Captura de video.
- Visualización de detecciones.
- Dibujo de bounding boxes.
- Procesamiento de imágenes.
- Integración futura de detección de color.

Parte 1: Creación del ambiente virtual

Primero se creó un ambiente virtual para evitar conflictos de dependencias.

Creación del entorno

```
python3 -m venv ~/yoloenv
```

Activación del entorno

```
source ~/yoloenv/bin/activate
```

Cuando el entorno estaba activo aparecía:

```
(yoloenv)
```

Esto indica que todas las librerías se instalan únicamente dentro del entorno virtual.

Parte 2: Instalación de librerías

Instalación de Ultralytics

```
pip install ultralytics
```

Instalación de OpenCV

```
pip install opencv-python
```

Instalación de OpenCV contrib

Posteriormente también se instaló:

```
pip install opencv-contrib-python
```

Esto se hizo porque más adelante se planeó utilizar trackers.

Parte 3: Dataset inicial de señales

Inicialmente se utilizó un dataset basado en señales de tránsito.

Las clases utilizadas fueron:

stop
straight
turn_right
turn_left
speed_limit_30

Y

traffic_light

Parte 4: Entrenamiento de YOLO

El entrenamiento se realizó usando Ultralytics.

Comando de entrenamiento

```
yolo task=detect mode=train \  
model=yolo11n.pt \  
data=gtsrb_det_plus_real/data.yaml \  
epochs=30 \  
imgsz=416 \  
batch=8    // esto se lo fui cambiando, iba a iniciar con 20 batches, pero mi compu murió
```

Explicación teórica del entrenamiento

model=yolo11n.pt

Se utilizó YOLO11 nano como modelo base, porque:

- consume menos recursos,
- funciona en tiempo real,
- para robótica móvil.

data.yaml

El archivo YAML contiene:

- rutas del dataset,
- clases,

- imágenes de entrenamiento,
- imágenes de validación.

epochs

Las épocas indican cuántas veces la red neuronal recorre el dataset completo.

Se utilizaron 30 épocas para permitir que el modelo aprendiera mejor las características de las señales. Mi compu no aguantó más de 30.

imgsz

La resolución de entrenamiento fue 416, que es un balance entre:

- velocidad,
- precisión,
- consumo de VRAM.

batch

El batch size controla cuántas imágenes procesa la red neuronal simultáneamente.

Se utilizó batch=8 para evitar problemas de memoria.

Parte 5: Agregado de semáforos

Después se agregó una nueva clase:

traffic_light

Para esto se descargó un dataset desde

Roboflow. <https://universe.roboflow.com/traffic-light-be5sc/traffic-light-1wdof>

Parte 6: Integración de datasets

El dataset descargado fue integrado al dataset original.

Pero, el dataset de Roboflow tenía IDs diferentes.

Por ejemplo:

0 = red

1 = green
2 = yellow

Pero en el dataset principal:

5 = traffic_light

Entonces se modificaron los labels.

Corrección de labels

Se utilizaron comandos sed para cambiar las clases.

Ejemplo

```
sed -i 's/^0 /5 /g' train/labels/*.txt
```

Esto reemplazaba:

0

por:

5

Parte 7: Entrenamiento combinado

Después de combinar datasets se volvió a entrenar el modelo.

El modelo final aprendió:

stop
straight
turn_right
turn_left
speed_limit_30
traffic_light

Resultados del entrenamiento

Durante la validación se observaron métricas altas.

Ejemplo:

- mAP50
- Precision
- Recall

Estas métricas sirven para evaluar el rendimiento.

Precision

Indica qué tan correctas son las detecciones.

Recall

Indica qué tantos objetos reales fueron encontrados.

mAP

Mean Average Precision.

Es la métrica principal para evaluar detectores de objetos.

Parte 8: Uso de doble YOLO

Posteriormente se decidió utilizar dos modelos simultáneamente.

Modelo 1

Modelo personalizado:

runs/detect/train-4/weights/best.pt

Detectaba:

- señales,
- semáforos.

Modelo 2

Modelo default:

yolo11n.pt

Detectaba obstáculos.

Se usaron dos modelos para no entrenar de nuevo el otro, y poder afectar su efectividad. Además:

- el modelo preentrenado ya sabe detectar obstáculos, el de yolo ultralytics,
- el modelo personalizado se especializa en señales, el de la base de datos y las fotos.

= un sistema híbrido.

Obstáculos seleccionados

No se utilizaron todas las clases de YOLO.

Solo:

- car
- bicycle
- motorcycle
- bus
- truck

Se intentó utilizar person, pero siempre hacía la detección, por lo que para evitar que siempre se detenga, person y chair fue removido.

Parte 9: Sistema de prioridades

Después se implementó un sistema de prioridades. Estas prioridades fueron cambiadas con respecto a un score asignado a cada clase.

Fórmula del score

Se utilizó:

$\text{score} = \text{prioridad} \times \text{área} \times \text{confianza}$

Explicación teórica

Prioridad

Cada clase tiene una importancia diferente.

Por ejemplo:

traffic_light = 300

stop = 260

Área

El área del bounding box sirve como aproximación de distancia.

Si la caja es más grande:

- probablemente el objeto está más cerca,
- el robot debe prestarle más atención.

Parte 10: Suavizado del score

El score cambiaba mucho entre frames.

Esto ocurría porque:

- el bounding box cambia ligeramente,
- la iluminación cambia,
- el autofocus cambia,
- la confianza cambia.

Entonces se implementó un suavizado.

Fórmula de suavizado

Se utilizó:

$\text{score_suavizado} = \alpha * \text{score} + (1-\alpha) * \text{score_anterior}$

Esto funciona como un filtro.

Permite:

- reducir ruido,
- disminuir parpadeos,

Se utilizó:

$\alpha = 0.10$, a prueba y error

Parte 11: Reducción de parpadeo

Inicialmente las detecciones desaparecían entre frames, porque:

- YOLO solo corría cada ciertos frames,
- pero OpenCV mostraba frames limpios.

Entonces se implementó memoria de frame.

entonces, se guardó el `ultimo_frame_out`

Y durante frames omitidos se mostraba:

`ultimo_frame_out`

Reduciendo significativamente el parpadeo.

Arquitectura final del sistema

El sistema final quedó:

Camara

↓

YOLO señales

↓

YOLO obstáculos

↓

Sistema de prioridades

↓

Selección de objetivo prioritario

↓

Visualización

↓

Futura integración ROS2

